Universidad Autónoma de Guadalajara
Campus Tabasco

Departamento Académico de
Ciencias Sociales y Humanidades

[Tipo de Actividad: Ensayo, Resumen, etc.]
Reporte de practica 2.

Programa
Ing. De software y Minería de Datos

Asignatura
Programación Orientada a Objetos.

Presenta
Daniel Ernesto Garrido Sánchez.
Matrícula 5211038

Docente
Manuel Balderas Victoria.

Villahermosa, Tabasco, México
16/10/2026

Introducción.

La programación orientada a objetos permite organizar un programa mediante clases y objetos, facilitando la reutilización del código y la representación de elementos del mundo real. En esta práctica se desarrollan cinco ejercicios en C++ enfocados en la herencia, el polimorfismo, la sobrecarga de operadores, la sobrecarga de métodos y el uso de colecciones con iteradores.

El propósito es reforzar estos conceptos mediante programas pequeños y comprobar su funcionamiento. Para documentar la práctica, cada ejercicio debe incluir código fuente, captura de ejecución, enlace al repositorio de GitHub y una reflexión personal.

Objetivo General.

Aplicar conceptos fundamentales de programación orientada a objetos en C++, desarrollando programas que permitan comprender las relaciones entre clases, el comportamiento polimórfico y la reutilización de operaciones mediante la sobrecarga.

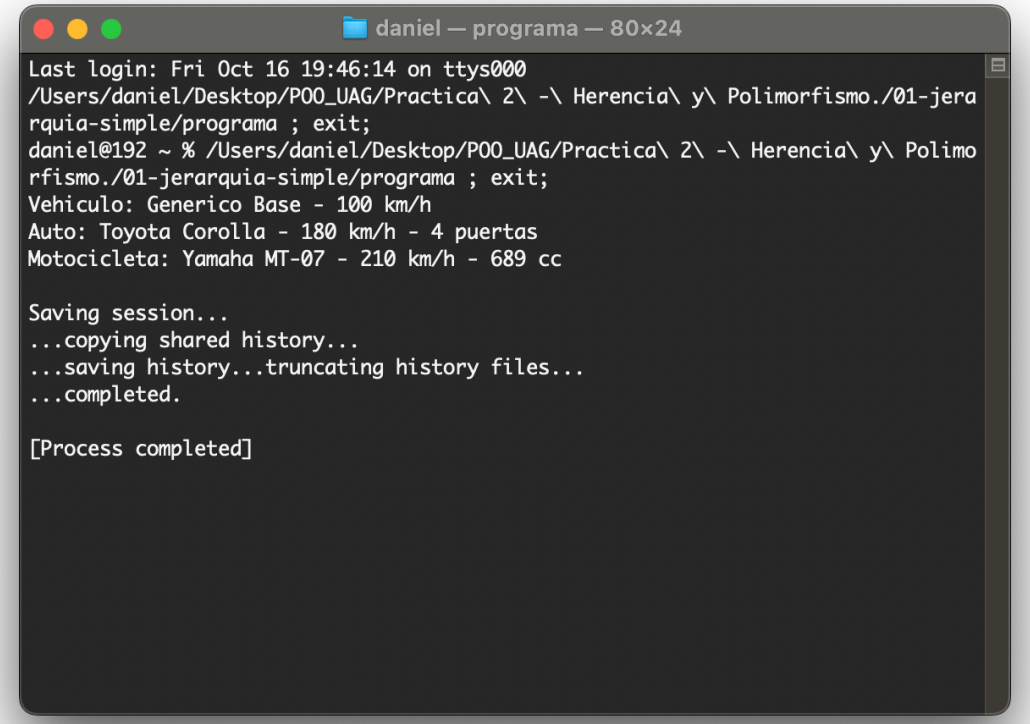
Desarrollo de los ejercicios.

Ejercicio 1: Jerarquía-simple.

Link:

https://github.com/DanielErnestoGS/POO_UAG/tree/main/Practica%202%20-%20Herencia%20y%20Polimorfismo./01-jerarquia-simple

Captura de pantalla:



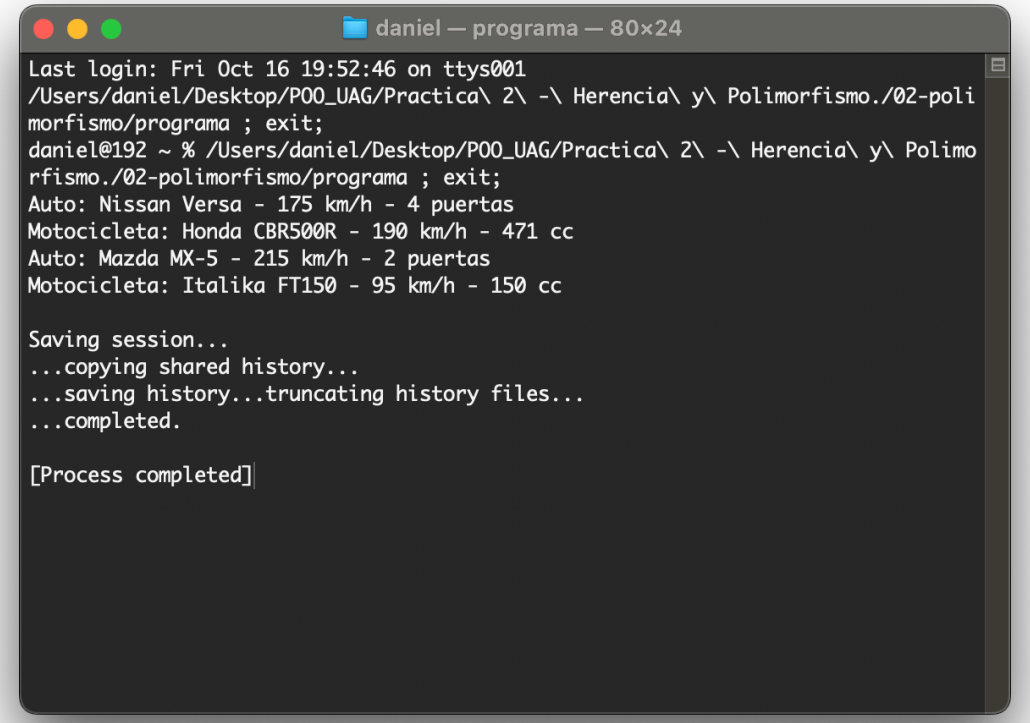
Reflexión: En este ejercicio comprendí mejor para qué sirve la herencia y por qué conviene colocar en una clase base los datos que varios objetos tienen en común. Por ejemplo, la marca, el modelo y la velocidad máxima describen a cualquier vehículo, mientras que otros atributos permiten distinguir un automóvil de una motocicleta. La parte que requiere más atención es decidir qué pertenece a la clase general y qué debe quedarse en cada clase derivada; si esa separación no se planea bien, el código puede quedar repetido o poco claro. También es necesario que cada clase implemente su propia versión de mostrarInfo() para presentar sus características. Este ejercicio me ayuda a ver que diseñar las clases antes de escribir todo el programa facilita organizarlo y modificarlo después.

Ejercicio 2: Polimorfismo.

Link:

https://github.com/DanielErnestoGS/POO_UAG/tree/main/Practica%202%20-%20Herencia%20y%20Polimorfismo./02-polimorfismo

Captura de pantalla:



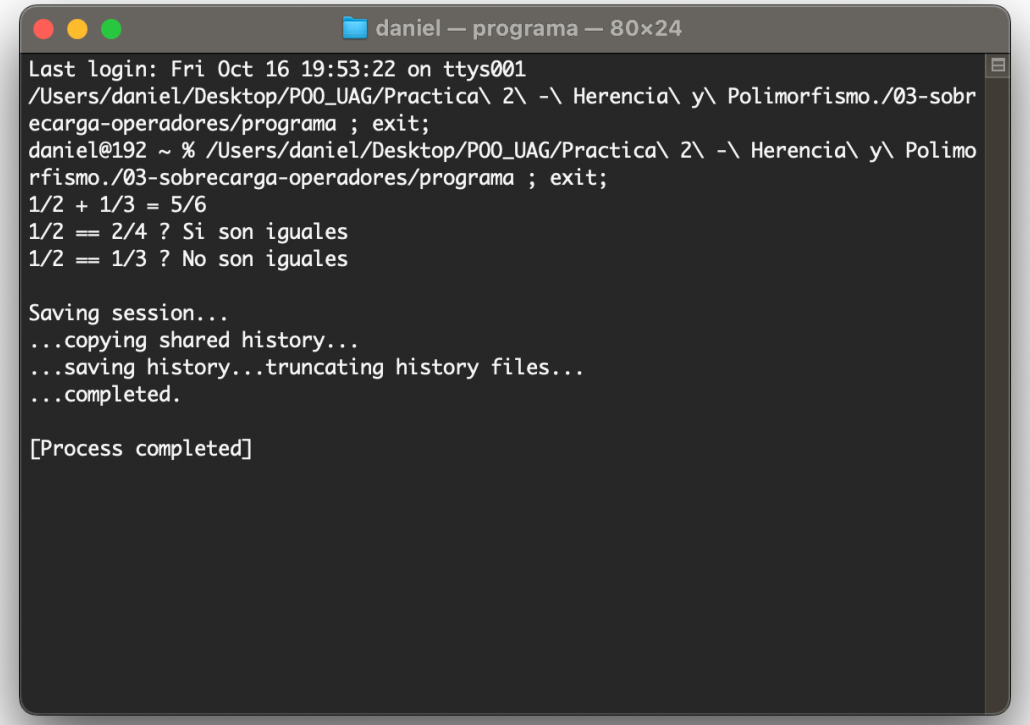
Reflexión: Este ejercicio me permite distinguir entre tener objetos de distintas clases y aprovechar realmente el polimorfismo. La idea es que la colección pueda manejar los vehículos mediante el tipo base, pero que al llamar a mostrarInfo() se ejecute la versión correspondiente al automóvil o a la motocicleta. Para conseguirlo, no basta con que los métodos tengan el mismo nombre: hay que declarar correctamente la función virtual en la clase base y sobrescribirla en las clases derivadas. Esta relación puede resultar confusa al principio, especialmente cuando se usan punteros o referencias, pero observar las salidas de cada objeto ayuda a comprobar qué método se ejecutó. Lo importante que me llevo es que el polimorfismo permite trabajar con tipos diferentes de una manera uniforme sin perder su comportamiento particular.

Ejercicio 3: Sobrecarga-operadores.

Link:

https://github.com/DanielErnestoGS/POO_UAG/tree/main/Practica%202%20-%20Herencia%20y%20Polimorfismo./03-sobrecarga-operadores

Captura de pantalla:



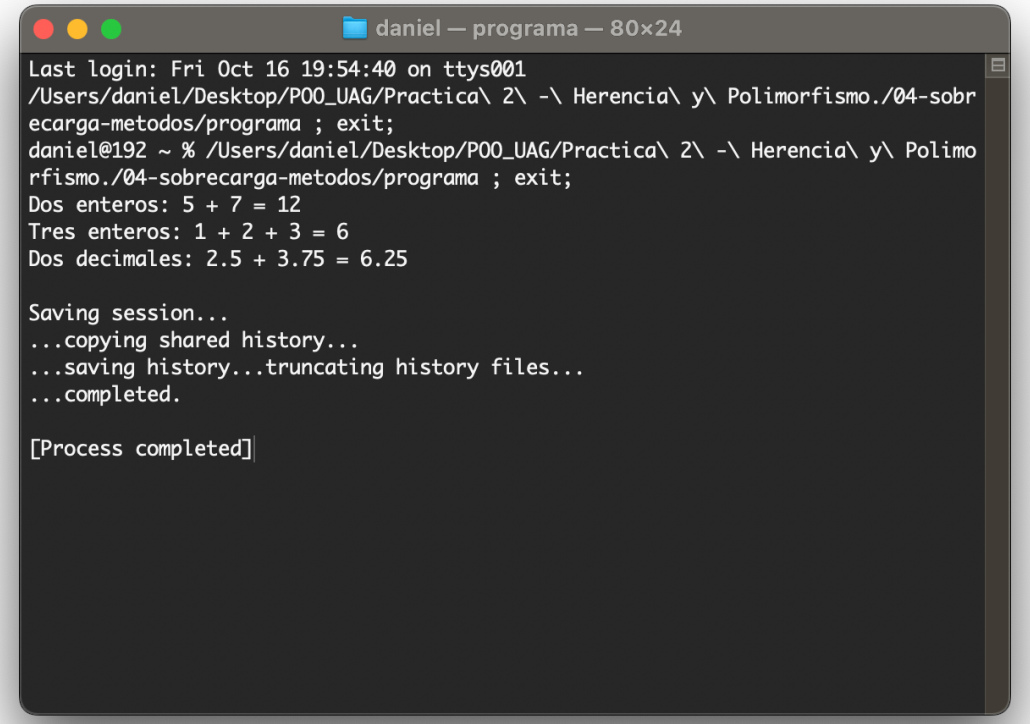
Reflexión: Con este ejercicio entendí que la sobrecarga de operadores permite que una clase tenga operaciones más naturales para quien utiliza sus objetos. En lugar de crear nombres de funciones diferentes para cada operación, se puede usar el operador + para sumar fracciones y == para comprobar si dos fracciones representan el mismo valor. La dificultad no está solamente en escribir los operadores, sino en respetar las reglas matemáticas: para sumar se deben considerar los denominadores y, para comparar, dos fracciones pueden ser equivalentes, aunque tengan numeradores y denominadores distintos. También se debe evitar un denominador igual a cero, porque una fracción así no es válida. Probar ejemplos como 1/2 y 2/4 permite revisar si la comparación funciona como se espera y detectar errores en la lógica.

Ejercicio 4: Sobrecarga-métodos.

Link:

https://github.com/DanielErnestoGS/POO_UAG/tree/main/Practica%202%20-%20Herencia%20y%20Polimorfismo./04-sobrecarga-metodos

Captura de pantalla:



Reflexión: En este ejercicio aprendí que una clase puede ofrecer varias funciones con el mismo nombre cuando sus parámetros permiten diferenciarlas. En Calculadora, sumar() puede recibir dos enteros, tres enteros o dos números decimales; el compilador decide qué versión utilizar según la llamada. Una complicación es recordar que cambiar únicamente el nombre de los parámetros no crea una sobrecarga distinta: debe cambiar la cantidad o el tipo de parámetros. También es importante revisar los tipos utilizados para que una operación con decimales no pierda precisión innecesariamente ni se confunda con una versión destinada a enteros. Probar cada variante por separado ayuda a comprobar que se está llamando a la función correcta. Este ejercicio muestra cómo reutilizar un

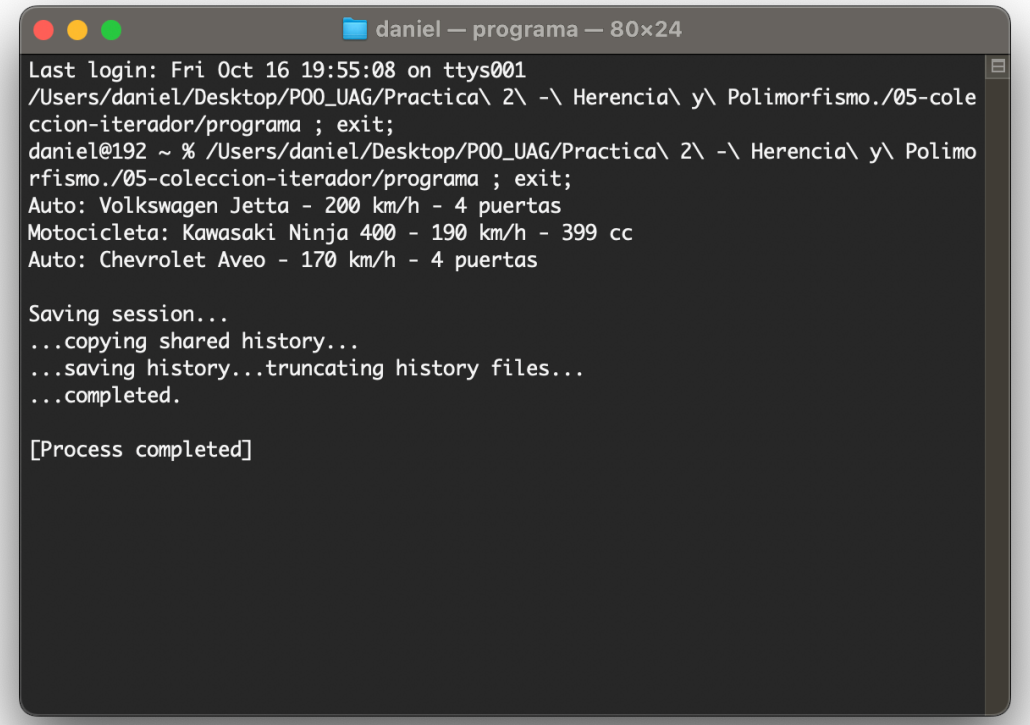
nombre de método sin tener que crear una función completamente diferente para cada caso.

Ejercicio 5: Colección-iterador.

Link:

https://github.com/DanielErnestoGS/POO_UAG/tree/main/Practica%202%20-%20Herencia%20y%20Polimorfismo./05-coleccion-iterador

Captura de pantalla:



Reflexión: Este ejercicio reúne varios conocimientos anteriores porque no solo se guardan vehículos en una colección, sino que también se recorren para mostrar su información. Utilizar un vector permite almacenar varios elementos y un iterador permite avanzar por ellos sin tener que acceder manualmente a cada posición. La parte que exige más cuidado es comprender dónde inicia y dónde termina el recorrido: si la condición del ciclo está mal escrita, pueden quedar elementos sin mostrar o intentarse acceder a una

posición que no corresponde. Además, si los elementos se manejan como objetos de la clase base, hay que mantener la estructura compatible con el polimorfismo para que se muestre la información adecuada. Esta práctica me ayuda a entender que las colecciones son útiles cuando la cantidad de objetos puede crecer y que los iteradores ofrecen una forma ordenada de procesarlos.

Complicaciones generales:

Una dificultad común a los cinco ejercicios es pasar de conocer la definición de un concepto a saber cuándo y cómo aplicarlo en código. En la herencia, la decisión importante es separar los datos generales de los específicos; en el polimorfismo, entender por qué una llamada mediante el tipo base puede ejecutar una implementación derivada; y en las sobrecargas, respetar las reglas de los parámetros y de los operadores. También hay que prestar atención a detalles que pueden parecer pequeños, como validar el denominador de una fracción, distinguir enteros de decimales y establecer correctamente el límite de un iterador.

Una manera de trabajar estas dificultades es construir cada programa por etapas: primero definir las clases y sus responsabilidades, después implementar los métodos y finalmente probar casos concretos. Revisar los mensajes del compilador ayuda a localizar errores de sintaxis o de tipos, mientras que probar diferentes entradas permite detectar errores de lógica que no siempre impiden compilar. Mantener cada ejercicio en su propia carpeta y acompañarlo con una captura facilita revisar qué se hizo y comprobar que el resultado coincide con lo esperado.

Conclusión:

La práctica permite reforzar que la programación orientada a objetos no consiste únicamente en crear clases, sino también en establecer relaciones entre ellas y elegir una estructura adecuada para cada problema. La herencia permite reutilizar características comunes; el polimorfismo hace que los objetos respondan de manera distinta a una misma llamada; y la sobrecarga adapta operadores y métodos a distintos datos o parámetros.

Las complicaciones más importantes están en comprender cuándo utilizar cada concepto, definir correctamente los métodos sobrescritos, realizar las operaciones con fracciones y

distinguir las versiones de un método sobrecargado. Además, las colecciones y los iteradores exigen cuidar la sintaxis y el recorrido de los elementos. Para superar estos problemas, es útil compilar con frecuencia, probar cada función por separado y revisar los resultados antes de continuar. Las capturas y el repositorio permiten presentar evidencias claras del trabajo.